

BonaFlow

Find food faster. Keep every station flowing.

FREEZE
16:00

DEMOS
16:30

PITCHES
17:15

TEAM
2 people

01 Orientation

Read this page before you write a line of code. It is the whole strategy in one screen.

The core transformation – this must be visible on stage

ONE SENTENCE BECOMES FIVE OPERATIONAL CHANGES

"Vegan Thai Curry is almost finished, and around 20 guests are waiting."

→ Station B availability drops · a **high-priority replenishment task** is created · the ops dashboard updates · **guests are redirected to Station C** · a multilingual announcement is generated.

The insight to lead with

The gap is not sensing. The gap is distribution. The person behind Station B already knows the vegan curry is nearly gone. That fact is worth a great deal to 250 guests and to the kitchen – and today it travels no further than the two metres in front of them.

Three seconds of a staff member's voice becomes a live redirect for every guest in the room, in their own language.

The problem, stated so a caterer nods

Guests	Long queues at one station while another stands quiet. Can't locate vegan or vegetarian choices. International guests miss announcements entirely.
Catering teams	Food runs out without operations knowing. Staff coordinate through fragmented conversations. Replenishment is late. Surplus at one station, shortage at another.

Static signs and printed menus cannot respond to live conditions. BonaFlow creates one shared real-time view for guests and catering teams.

Three modes, one app, no authentication

A plain demo-mode selector: **Guest · Staff · Operations**. No login screen, no password, no role picker.

WHY A MODE SELECTOR AND NOT THREE APPS

It is the reliability path – if the second device dies on stage you switch modes on one phone and the demo still runs. **But demo on two devices when you can:** the guest view changing *untouched* while you hold the staff phone is the single strongest moment in the pitch.

FOUR WAYS THIS DIES ON STAGE

- **The live update doesn't fire.** Poll at 3 seconds, not realtime magic – and build the manual override at 15:00 (\$05).
- **Waiting on the model in front of the room.** Quick actions change state with no AI in the path. Pre-generate the announcements.
- **Two devices, two people, one narration.** Rehearse the handoff twice.
- **Claiming to detect queues.** You don't, and you don't need to. Say so – it is the stronger claim.

02 The three views

Guest finds food. Staff reports reality. Operations sees everything.

Guest – tabs: Stations · For You · Updates

View all stations, queue status and availability · filter by dietary preference · get a recommended station · read and hear live announcements · a simple location label (“by the stairs”, not a map route) · mark that you are joining a queue.

Dietary filters – support only: All · Vegan · Vegetarian · Gluten-free option · Halal option.

⚠ THE LINE YOU MAY NEVER CROSS

- **Never claim a dish is allergy-safe.** Never render “safe” or “you can eat this.”
- **Filter on declared dietaryTags only.** The model never decides what a dish contains from its name, its photo, or its own knowledge. It filters a list.
- **No match → say so flatly:** “No station currently has a vegan option available.” Never soften, never offer a “probably fine” alternative.
- Always display: “Dietary labels are demonstration data. Guests must confirm allergens and ingredients with catering staff.”

Staff – tabs: Stations · Report · Tasks

Pick a station, then update by **voice**, **text**, **optional tray photo**, or **quick action**.

Quick actions one tap, no AI call	Stock running low · Item sold out · Replenishment arrived · Queue increasing · Queue cleared · Station temporarily closed
---	---

Build quick actions early. They exercise the entire downstream state machine with no OpenAI in the path – they are your latency insurance *and* your fallback *and* your 14:15 gate.

CONFIRM BEFORE APPLY – MANDATORY

The staff member reviews the extracted update in plain language and confirms before shared data changes. Edit affordance on every field, plus Cancel.

This is not friction, it is the product being honest – and it helps the demo, because the judges watch the extraction land before it takes effect.

When the photo and the human disagree, the human wins – and you show it. A tray can look full because it was just restacked with something else; the person standing there knows. The UI says “photo suggested available; staff reported low – using staff.” That one line is worth more than any accuracy claim you could make.

Operations – tabs: Overview · Alerts · Activity

All stations · status · queue level · available dishes · low-stock items · active alerts · replenishment tasks · most crowded station · recommended redirection · recent staff updates · **timestamp of the last update**.

 Green	Available
 Orange	Running low or busy
 Red	Sold out, closed or urgent
 Grey	No recent update – build this one. A station nobody has reported on is not “fine”, it is unknown, and showing that is more credible than a board that is green everywhere by default.

03 Architecture & the AI contract

Model reads. Code decides. Closed sets everywhere.

Surface – two viable paths, gated by the clock not the hardware

You have **two iPhones and one Android**, so you can build and demo cross-platform properly, and Bilt's iOS-only streamed simulator is an asset rather than a liability. **Test on both platforms from the first screen** – cross-platform bugs cost minutes now and an hour at 15:30.

Bilt / React Native	The partner prize, real native camera and mic. Phased prompts in BILT-PROMPT.md . Generate a real RN app, not a web wrapper.
Next.js PWA	No build step, no install friction for the room scanning your QR. Also cross-platform – Android Chrome and iOS Safari both give camera and mic.

THE BILT GATE, AND THE ONE IOS TRAP THAT ENDS DEMOS

If Phase 1 is not rendering on a real phone by 12:45, drop Bilt, say nothing about it on stage, and go all-in on the PWA.

⚠ **iPhone ring/silent switch**. With it on silent, audio playback is **completely silent with no error** unless the app sets `playsInSilentModeIOS`. Set it in code *and* check the physical switch before you present. On the web path, iOS Safari records **mp4/aac, not webm** – never hardcode the mime type.

Routes and state

```
/           mode selector           /ops  operations dashboard
/e/[eventId] guest navigator       /api/* server-side only
/staff      staff update

GET /api/event/:id      -> { event, stations[], dishes[], alerts[], tasks[] }
POST /api/staff-update  -> { stationId, transcript?, imageBase64?, quickAction? }
POST /api/apply         -> { extraction } -> { ok, changed[] }
GET /api/guest?diet=vegan -> filtered + ranked
```

Entities: Event · Station · Dish · StaffUpdate · Alert · ReplenishmentTask.

Supabase, or **seeded local state** the moment it costs more than 25 minutes. Realtime if free; **otherwise poll every 3 seconds and stop thinking about it** – polling reads as real-time to an audience and never fails mysteriously. **Never in-memory**: serverless drops it between requests, the phone writes, the laptop reads nothing, and you lose twenty minutes finding out why.

THE SEQUENCING RULE THAT UNBLOCKS EVERYTHING

Agree the data model **together, on paper, ten minutes**, before either of you types. Then **ship a stub POST /api/staff-update returning hardcoded valid JSON within twenty minutes**. The UI wires against the stub immediately and never waits.

The AI contract

One server-side call per staff update. Structured Outputs, `strict:true`. Keys stay server-side. Inputs: transcript · optional tray photo · **the closed list of this event's station and dish ids**.

```
{
  "stationId": "station-b",      "stationName": "Green Kitchen",
  "dishId": "vegan-thai-curry",  "dishName": "Vegan Thai Curry",
  "availability": "available | low | sold_out | uncertain",
  "queueLevel": "low | medium | high | unknown",
  "reportedGuestCount": 20,
  "issueType": "low_stock | sold_out | queue | closure | resolved | other",
}
```

```

    "priority": "low | medium | high | urgent",
    "reportedFacts": ["string"],
    "aiInferences": ["string"],
    "recommendedAction": "string",
    "recommendedAlternativeStationId": "station-c",
    "guestAnnouncement": "string",
    "confidence": 0.0
  }

```

- **Closed sets.** `stationId` and `dishId` must be ids from the supplied lists – never invented. This single constraint removes most of the hallucination class you would otherwise spend the afternoon debugging.
- **Reported facts and AI inferences never merge.** Two visually distinct rows. Every inference carries its confidence.
- **Never present uncertain visual analysis as fact.** Photo-only and unsure → it is an inference, not a fact.
- **Never invent a count.** `reportedGuestCount` is only ever a number a human said; otherwise null. No portions, no grams, no wait-time prediction.
- Validate server-side. On invalid output: keep the transcript, **apply nothing**, show the correction UI.

THE MODEL PROPOSES. CODE DECIDES. SAY THIS ON STAGE.

The model returns `recommendedAlternativeStationId`. **Validate it before you ever show it** – does that station actually have a matching dish marked available? If the suggestion fails the check, the deterministic rule wins and only code can return “nowhere has this.”

“The model reads. The rules decide. That’s why it’s auditable.” A caterer who has to explain a decision to a client hears that clearly.

```

resolveRedirect(dish, fromStation, modelSuggestion):
  candidates = stations with a matching-dietaryTags dish, availability "available"
  drop fromStation
  rank by queueLevel (low < medium < high < unknown), then proximity
  if modelSuggestion in candidates -> use it
  else if candidates                -> use candidates[0], log the override
  else                              -> null, "No station currently has this."

```

04 Demo event & build order

Future of Work Summit Berlin · Delta Campus · 250 guests · lunch 12:30–14:00. All of it labelled as demonstration data.

STATION	DISHES & DIETARY	INITIAL STATE
A · Mediterranean Kitchen <i>main hall, left</i>	Mediterranean Chicken Bowl halal · Roasted Vegetable Couscous vegetarian	queue medium · good
B · Green Kitchen <i>by the stairs</i>	Vegan Thai Curry vegan , gluten_free · Tofu Rice Bowl vegan	queue high · curry LOW
C · Pasta Corner <i>back room</i>	Seasonal Vegetable Pasta vegan · Tomato Basil Pasta vegetarian	queue low · good
D · Grab & Go <i>near the entrance</i>	Sandwiches · Fruit · Salads · Drinks	queue low · good

ON REAL DATA – THE DELIBERATE CALL

The seeded event is the **demo path**: a scripted two-minute demo needs guaranteed initial state, and you cannot script on data you have not collected yet.

If you get ahead, capture today’s actual lunch as a *second* seed with an event switcher – real station and dish names, three tray photos (full / half / nearly empty). Then you get to say: “*and we ran this on your actual lunch today.*” The judges ate that food. **Opportunistic, never a dependency.** Ask before photographing anyone; quote people exactly.

Build order – nothing below the line until the line works

- 1

Guest station list
- 2

Dietary filtering
- 3

Staff update form + **quick actions**
- 4

Confirmation screen
- 5

Shared station-state update – the journey is real here
- 6

Operations dashboard
- **14:15 GATE** – the full loop runs on quick actions alone, with no OpenAI in the path. If it doesn’t, you are behind: go to the floor (§05).
- 7

OpenAI extraction
- 8

ElevenLabs announcement (EN, then DE)
- 9

Camera input
- 10

Visual polish

The clock

12:20 Stub shipped, deployed, a live URL exists

12:45 **Bilt go/no-go** – Phase 1 on a real Android phone, or drop it

14:15 **HARD GATE** – staff action → guest view changes untouched. Ugly is fine.

15:00 Freeze the hero path · **build the manual override** · pre-generate announcements · test scrpcy

15:30 **Record the backup video**

16:00 **FREEZE.** Rehearse the two-device dance twice. Test audio in the room.

Real-time behaviour – on confirmation, in this order

save the update → change dish availability → change station status and `lastUpdatedAt` → create the alert → create the replenishment task → recalculate the recommended station → update Guest view → update Operations view → generate the announcement.

05 Demo, fallbacks, pitch

Under two minutes. Step 11 is the entire pitch.

	YOU SAY / DO	ON SCREEN
1	"At large events the problem is rarely that there is no food..." QR on slide one – ask the room to scan.	Guest view, four stations
2	Tap the Vegan filter.	B and C match
3	"Station B is the closest match – but look at the queue."	B: queue high
4	"So it already suggests Station C."	For You: Station C
5–6	Switch to Staff View , select Station B.	Station picker
7	" <i>Vegan Thai Curry is almost finished, and approximately 20 guests are waiting.</i> "	Recording
8–9	"Here's what it understood – and nothing changes until I confirm." Tap Confirm.	Confirmation: facts vs inferences, separate rows
10	Station B goes orange → red. Task appears.	Red status · replenishment task, high
11	Do not touch the guest device. Let it change by itself. Count to two out loud.	Guest view updates in the room's hands
12–13	Vegan guests redirected to C. ElevenLabs announces, EN then DE.	Announcement
14	"And operations already has the alert."	Ops: alert + latest update

RECOVERY – BUILD THIS AT 15:00, NOT AT 16:20

The manual override: a hidden control that forces the scripted state change with no network at all. If the parse hangs, tap it and say "*the parse is real, I'm firing the cached one.*" Honest, and the demo continues.

Devices: iPhone #1 = **guest** (mirrored, the screen that changes untouched) · iPhone #2 = **staff** (in hand, off-screen) · Android = **operations board** and cross-platform proof.

Mirroring, set up at 15:00. iPhone → Mac: **QuickTime → File → New Movie Recording → arrow beside record → select the iPhone** (needs a cable and tapping Trust; no wifi involved). Android → Mac: **scrcpy** over USB with USB debugging ON. Both need a **data-capable** cable – charge-only cables are common and fail silently. Phone hotspot as the primary network.

Fallbacks – all required

QR scanning fails	A button: Enter Demo Event
Microphone denied	Text input, pre-filled with the staff line
ElevenLabs fails	Pre-generated audio, or the announcement as text
OpenAI fails	A clearly labelled cached demo result
Backend fails	Local seeded state, simulated within the session
The live build fails	Tested backup build and the screen recording

Cut order

German announcements → tray-image analysis → QR scanner → replenishment assignment → persistent Supabase → dynamic AI recommendation (*the deterministic rule stands alone*).

Never cut: guest station view · dietary filter · staff update · status change · alert · guest redirection · operations dashboard · **the manual override** · **the backup recording**.

THE FLOOR – THE SMALLEST DEMO THAT STILL WINS

One laptop, two browser windows. Staff window: quick-action **"Item sold out."** Guest window: watch it turn red, re-rank, and hear the announcement.

"Two windows, one laptop – the loop is real, the transport is simple." **A working loop beats a beautiful app with a broken one.** Know this floor so you don't panic at 15:00.

Do not build

Payments · ordering · accounts · GPS or indoor positioning · camera crowd counting · facial recognition · tray-volume measurement · wait-time prediction · real B&B integration · inventory management · delivery logistics · push infrastructure · multi-event · App Store deployment · ratings or gamification. **Not a complaint box** – guests report availability, not grievances. **Not a nutrition tracker. No allergen or calorie estimation from a photo, ever.**

Pitch – three minutes

Hook: *"At large events, the problem is rarely that there is no food. The problem is that guests and catering teams don't know what's happening across every station."* → **Problem** (queues, shortages, dietary confusion, fragmented comms) → **Product** → **Live demo** → **Business value** (guest experience · faster response · better distribution · fewer shortages · directional waste reduction · structured event data – **no euro figures, you cannot substantiate one**) → **Technology**, one sentence each, then *"the model reads, the rules decide – that's why it's auditable"* → **Expansion** (conferences, campuses, festivals, stadiums, universities, trade fairs) → **Close:** *"BonaFlow turns every catering station into a connected, responsive part of the event."*

Q&A POCKET

- **"Do you detect queues?"** → *"No, and we don't claim to. Your staff already know. We move what they know to everyone who needs it, in three seconds."*
- **"Is this only for events?"** → B&B sell event catering explicitly. **Ask a B&B person at lunch how big that line is** and use their number. Don't guess it.
- **"What's next?"** → *"Today BonaFlow keeps the event flowing. The same stations know what ran out and what didn't – after the event, that becomes what to cook more of next time."*

Design tokens



#F0F0F0 warm off-white	#14171A ink	#5A6169 soft	#0F766E green · available	#F08A4B coral · low	#B4432B red · urgent	#D8DDD6 grey · no update
---------------------------	----------------	-----------------	------------------------------	------------------------	-------------------------	-----------------------------

Premium but friendly. **Live, clear, fresh, operational, trustworthy, immediately understandable.** Rounded station cards, large status indicators, simple food icons, clear dietary tags, large tap targets, minimal text, bottom navigation, portrait-first. **Every status value and timestamp in a monospace face** – monospace-for-data is an honesty signal and costs nothing. **Colour carries meaning only in station status**, so green/amber/red reads instantly from the back of the room. **Do not copy Bella&Bona's visual identity.** Show, visibly: *"Independent hackathon prototype created for demonstration purposes."*

Success criteria – must be true at 16:00

- ☐ A judge understands the problem in 15 seconds
 - ☐ Four stations · dietary filter on **declared** tags
 - ☐ Recommends an alternative station
 - ☐ Staff submit voice or text; quick actions work with no AI
 - ☐ Validated extraction against the **closed** id lists
 - ☐ Staff **confirms** before state changes
 - ☐ **Guest view changes without being touched**
 - ☐ Facts vs inferences visibly separate, with confidence
 - ☐ Alert + task appear; grey state and timestamps visible
 - ☐ ElevenLabs plays the announcement
 - ☐ Manual override tested · **backup video recorded**
 - ☐ Disclaimers visible · full flow under two minutes
-

Dish images

Ten CC-licensed placeholders sit in `assets/dishes/` so nothing renders an empty slot while you build. **Filenames are the contract** – reference images by filename, then swapping a placeholder for a real photo is a file overwrite, not a code change. **Shoot the real dishes at lunch and overwrite them**, hero dish (`vegan-thai-curry.jpg`) first. Portrait, one dish per frame, no hands or faces, then `sips -Z 900` before committing. Details and licensing in `assets/dishes/README.md`. **Do not substitute Bella&Bona's own product photography** – copyrighted, and it contradicts your "not affiliated" disclaimer.

BonaFlow · 8x × Bella&Bona Mobile Hackathon · 1 August 2026 · Martin & Vijay · Spec of record: `BONAFLOW-MASTER-PROMPT.md` · Bilt phases: `BILT-PROMPT.md` – copy from the files, never from this PDF.